

Node.js I/O Queue and I/O Polling - Interview Notes

What is the I/O Queue?

The I/O (Input/Output) Queue is one of the queues in the Node.js Event Loop that handles asynchronous I/O operations like file system operations, network requests, and database queries.

How to Add Functions to I/O Queue

Use async methods from built-in Node.js modules:

- `fs.readFile()` - Reading files
- `fs.writeFile()` - Writing files
- Network operations
- Database operations

Example:

```
const fs = require("fs");

fs.readFile(__filename, () => {
  console.log("File read complete");
});
```

Priority Order in Event Loop

The Event Loop executes queues in this order:

1. **Microtask Queues** (Highest Priority)
 - nextTick queue (`process.nextTick()`)
 - Promise queue (`Promise.resolve().then()`)
2. **Timer Queue**
 - `setTimeout()`
 - `setInterval()`
3. **I/O Queue** ← We're here
 - File operations
 - Network operations
4. **Check Queue**

- `setImmediate()`

5. Close Queue

- Close callbacks

Key Interview Points

1. Microtasks Execute Before I/O Queue

```
const fs = require("fs");

fs.readFile(__filename, () => {
  console.log("readFile");
});

process.nextTick(() => console.log("nextTick"));
Promise.resolve().then(() => console.log("Promise"));

// Output:
// nextTick
// Promise
// readFile
```

Why? Microtask queues have higher priority than I/O queue.

2. The `setTimeout(0)` Problem with I/O

Important Interview Question: What happens when you use `setTimeout(0)` with an I/O operation?

```
setTimeout(() => console.log("setTimeout"), 0);

fs.readFile(__filename, () => {
  console.log("readFile");
});

// Output is UNPREDICTABLE! Could be either:
// setTimeout → readFile
// OR
// readFile → setTimeout
```

Why is it unpredictable?

- `setTimeout(0)` is actually converted to `setTimeout(1)` (minimum 1ms delay)
- When the event loop starts, it checks if 1ms has passed
- **If CPU enters timer queue at 0.05ms:** Timer hasn't elapsed yet → moves to I/O queue → executes `readFile` first
- **If CPU enters timer queue at 1.01ms:** Timer has elapsed → executes `setTimeout` first → then `readFile`

Key Takeaway: Order of execution between `setTimeout(0)` and I/O operations can NEVER be guaranteed.

3. Complete Priority Order

```
const fs = require("fs");

fs.readFile(__filename, () => {
  console.log("readFile");
});

process.nextTick(() => console.log("nextTick"));
Promise.resolve().then(() => console.log("Promise"));
setTimeout(() => console.log("setTimeout"), 0);

// Add long loop to ensure setTimeout has elapsed
for (let i = 0; i < 2000000000; i++) {}

// Guaranteed Output:
// nextTick
// Promise
// setTimeout
// readFile
```

Order: Microtasks → Timer → I/O

I/O Polling - The Critical Concept

What is I/O Polling?

I/O Polling is the process where the Event Loop checks if I/O operations have completed BEFORE adding their callbacks to the I/O queue.

The Check Queue (setImmediate)

`setImmediate()` adds callbacks to the Check Queue, which comes AFTER the I/O queue in theory, but...

The Surprising Behavior

```
const fs = require("fs");

fs.readFile(__filename, () => {
  console.log("readFile");
});

setImmediate(() => console.log("setImmediate"));

// Output:
// setImmediate
// readFile ← Why does this come LAST?
```

Explanation - How I/O Polling Works

Step-by-step execution:

1. Call Stack Execution:

- `fs.readFile()` is called → I/O operation STARTS (not queued yet)
- `setImmediate()` is called → callback added to Check Queue

2. Event Loop Iteration 1:

- Check Microtask queues → Empty
- Check Timer queue → Empty
- Check I/O queue → **EMPTY!** (readFile hasn't completed yet)
- **I/O Polling Phase:** Event loop asks "Is readFile done?"
 - Answer: "Yes, just finished!"
 - readFile callback is NOW added to I/O queue
- Check Queue → Execute `setImmediate` callback → logs "setImmediate"

3. Event Loop Iteration 2:

- Check I/O queue → Execute readFile callback → logs "readFile"

Key Interview Point

Q: Why does setImmediate execute before readFile even though I/O queue comes before Check queue?

A: Because of I/O Polling:

- I/O callbacks are NOT queued immediately when the operation starts
 - They are only queued AFTER polling confirms the operation is complete
 - By the time polling completes and adds the callback to I/O queue, the event loop has already moved past it
 - The Check queue is next, so `setImmediate` executes first
 - readFile callback waits for the next iteration
-

Important Interview Questions & Answers

Q1: What is the priority of I/O queue in the Event Loop?

A: I/O queue has lower priority than Microtask queues and Timer queue, but higher priority than Check queue (in subsequent iterations).

Q2: Can you guarantee execution order between `setTimeout(0)` and I/O operations?

A: No! Because `setTimeout(0)` becomes `setTimeout(1)`, and depending on CPU timing, either could execute first.

Q3: Why might `setImmediate` execute before an I/O callback?

A: Because of I/O polling. I/O callbacks are only added to the queue after polling confirms completion. If polling happens after the I/O queue check, the callback waits for the next iteration, while `setImmediate` in the Check queue executes in the current iteration.

Q4: What happens when both I/O queue and Check queue have callbacks?

A: In the same iteration, I/O queue callbacks ALWAYS execute before Check queue callbacks. The polling issue only affects the first iteration.

Quick Memory Tips

Priority Order (Mnemonic: "Mike Takes Ice Cream"):

- **M**icrotasks (`nextTick`, `Promise`)
- **T**imers (`setTimeout`, `setInterval`)
- **I**O Queue (`fs.readFile`, `network`)
- **C**heck Queue (`setImmediate`)

I/O Polling Rule: "I/O callbacks need polling before queuing" - They're not queued until the operation completes and polling confirms it.

Common Mistakes to Avoid

✗ Assuming `setTimeout(0)` executes immediately ✗ Thinking I/O callbacks are queued when the operation starts ✗ Expecting predictable order between `setTimeout(0)` and I/O operations ✗ Forgetting that `setImmediate` can execute before I/O callbacks on first iteration

✓ Remember: Microtasks always first ✓ Remember: I/O polling happens between I/O and Check queues
 ✓ Remember: `setTimeout(0)` = `setTimeout(1)` ✓ Remember: Use long-running code to ensure timers elapse for predictable testing